JAVA & LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Lý thuyết diễn giải và ví dụ từng bước

Biên soạn: Phan Tiến Đạt

Biên soạn từ toàn bộ 9 PDF trong bản java-practice-main.zip được cung cấp ngày 11/11/2026: 181 trang nguồn.

Từ biến, điều kiện và mảng đến lớp, kế thừa, đa hình, quản lý danh sách và sơ đồ lớp UML.

Cách dùng tài liệu

Mỗi chủ đề đi theo trình tự: hiểu khái niệm → đọc mã → theo dõi dữ liệu → kiểm tra kết quả. Hãy tự đoán kết quả trước khi đọc phần giải thích, sau đó gõ lại ví dụ trong VS Code.

Các ví dụ được viết lại để làm rõ bản chất, không phải bản chép nguyên slide. Những phần lặp giữa các PDF được gộp. File bài tập bổ sung được chuyển thành các ví dụ vận dụng lý thuyết; tài liệu không nhằm giải toàn bộ bài tập cuối từng chương.

Quy ước mã nguồn

Mã dùng cú pháp cơ bản tương thích Java 17. Một khối chỉ có câu lệnh cần đặt trong main; một khối khai báo phương thức cần đặt trong lớp. Ví dụ ở các mục độc lập có thể dùng lại tên lớp, vì vậy không dán tất cả vào cùng một file. Các chương trình có public class Main được lưu thành Main.java.

Các chú thích “Bổ sung” dùng cho phần giải thích được thêm để hiểu slide. Phụ lục cuối chỉ rõ các lỗi cần tránh trong tài liệu nguồn.

Ghi nhận nguồn

Bộ slide được cung cấp trong kho java-practice của OwenDepTry. Trang bìa Chuong01a ghi giảng viên Trần Nguyễn Minh Hiếu. Bản diễn giải này phục vụ tự học và không thay thế tài liệu chính thức của học phần.

Bản đồ tài liệu và mục lục

Nguồn đã đọc

Mã	File PDF	Trang	Nội dung trong bản mới
S1	Chuong01a.pdf	14	Nền tảng, kiểu dữ liệu, nhập xuất
S2	Chuong01b.pdf	30	Điều khiển, mảng, chuỗi
S3	Chuong2_OOP.pdf	17	Tư duy hướng đối tượng
S4	Chuong03_LopVaDoiTuong.pdf	36	Lớp, biến, phương thức, constructor
S5	Chuong04_KeThua.pdf	21	Kế thừa, ép kiểu, is-a / has-a
S6	Chuong05_TinhDaHinh.pdf	24	Đa hình, abstract, final, lớp lồng
S7	Chuong06_HamChongVaTaoLopDanhSach.pdf	13	Nạp chồng, mảng đối tượng, CRUD
S8	ChuongVIII_ClassDiagram.pdf	19	UML và quan hệ giữa các lớp
S9	BaiTapBoSung.pdf	7	Xe, hình học, sạp chợ, ca sĩ

Trình tự học

01. Java căn bản: cấu trúc chương trình, kiểu dữ liệu, toán tử, nhập xuất. (trang 3)

02. Điều khiển: if, switch, vòng lặp, break, continue, return. (trang 7)

03. Mảng và chuỗi: tham chiếu, sao chép, xử lý dữ liệu. (trang 11)

04. Tư duy OOP: từ bài toán đến lớp và trách nhiệm. (trang 14)

05. Lớp và đối tượng: đóng gói, phạm vi, this, constructor, static, tham số. (trang 15)

06. Kế thừa: extends, super, ghi đè và ép kiểu. (trang 19)

07. Đa hình và trừu tượng: abstract, interface, final, lớp lồng. (trang 21)

08. Nạp chồng và danh sách: thêm, sửa, xóa, tìm kiếm. (trang 25)

09. UML: ký hiệu, bội số, liên kết, kết tập, hợp thành, phụ thuộc. (trang 29)

10. Vận dụng: phân tích bốn nhóm bài trong S9. (trang 32)

11. Ôn tập: các điểm hiệu chỉnh và câu hỏi tự kiểm tra. (trang 36)

Số trang ghi trong chú thích nguồn là thứ tự trang PDF, kể cả bìa. Một số slide ghi “Chương 4” ở nhiều file khác nhau; bản mới phân mục theo nội dung và tên file để tránh nhầm.

01 / Java căn bản

1.1. Chương trình bắt đầu từ đâu?

Nguồn: S1, trang 1-2, 4-5.

```java
public class Main {
    public static void main(String[] args) {
        int a = 5;
        int b = 3;
        System.out.println(a + b);
    }
}
```

Bước 1: Main là tên lớp chứa chương trình. Với cách tổ chức ví dụ này, lưu file thành Main.java, chú ý chữ hoa M.

Bước 2: main là điểm vào khi chạy lớp. public cho phép truy cập; static cho phép gọi mà chưa cần tạo một đối tượng Main; void nghĩa là phương thức không trả về giá trị. String[] args nhận các đối số dòng lệnh.

Bước 3: hai biến a và b nhận 5 và 3. Khi chạy println, Java tính a + b thành 8 rồi in ra màn hình và xuống dòng.

Dấu { } giới hạn thân lớp, thân phương thức hoặc khối lệnh. Dấu ; kết thúc nhiều loại câu lệnh như khai báo, gán và gọi phương thức; không đặt tùy tiện sau if hoặc for.

Định danh và ghi chú

Tên phân biệt hoa thường: diem và Diem là hai tên khác nhau. Nên dùng PascalCase cho lớp (SinhVien), camelCase cho biến/phương thức (diemTrungBinh) và UPPER_SNAKE_CASE cho hằng (MAX_SIZE). Không bắt đầu tên bằng chữ số, không dùng từ khóa. Ký tự _ được phép trong tên nhưng một dấu _ đứng riêng không phải tên hợp lệ trong Java 17.

```java
// Ghi chú một dòng
/* Ghi chú nhi□u dòng */
/** Mô t□ dùng đ□ tạo tài liệu Javadoc. */
final int SO_MON = 3;
```

final ngăn gán lại biến sau khi đã được gán. Hằng có tên giúp tránh việc rải số 3 ở nhiều nơi; nếu quy tắc thay đổi, chỉ cần sửa tại một chỗ.

Tự kiểm tra

Nếu đổi a thành 10 thì kết quả là 13. Nếu viết system.out.println thay cho System.out.println thì sai tên lớp; Java không tự sửa chữ hoa/chữ thường.

1.2. Kiểu dữ liệu: lưu cái gì và lưu được bao nhiêu?

Nguồn: S1, trang 3. Miền giá trị được hiệu chỉnh so với slide.

Kiểu	Kích thước giá trị	Ý nghĩa / khoảng chính
byte	8 bit	-128 đến 127
short	16 bit	-32.768 đến 32.767
int	32 bit	-2^{31} đến $2^{31} - 1$
long	64 bit	-2^{63} đến $2^{63} - 1$
float	32 bit	Số thực, khoảng 6-7 chữ số có nghĩa
double	64 bit	Số thực, khoảng 15-16 chữ số có nghĩa
char	16 bit	Một đơn vị mã UTF-16, 0 đến 65.535
boolean	Không quy định là 1 bit lưu trữ	true hoặc false

String là kiểu tham chiếu, không thuộc 8 kiểu nguyên thủy trên. char dùng dấu nháy đơn: 'A'; String dùng dấu nháy kép: "A". Một ký tự hiển thị, chẳng hạn một số emoji, có thể cần hai char.

```java
int tuoi = 21;
long danSo = 8_000_000_000L;
float diem = 8.5f;
double banKinh = 2.5;
char loai = 'A';
boolean dat = diem >= 5;
String ten = "Dat";
```

Hậu tố L tạo hằng long; f tạo hằng float. Mặc định một hằng số thực như 8.5 có kiểu double. Viết float x = 8.5; sẽ lỗi vì không tự thu hẹp double thành float.

Ép kiểu và tràn số

```java
int a = 7;
double b = a;       // 7.0, m□ rộng ki□u
int c = (int) 7.9;  // 7, b□ ph□n thập phân
long p = 50_000L * 50_000; // 2_500_000_000
```

Ở dòng cuối, hậu tố L làm phép nhân diễn ra ở kiểu long. Viết long p = 50_000 * 50_000; vẫn có thể sai: hai toán hạng int được nhân và tràn trước khi gán kết quả sang long.

Số thực được lưu gần đúng theo nhị phân; 0.1 + 0.2 có thể hiện thành 0.30000000000000004. Đừng dùng phép so sánh bằng tuyệt đối cho kết quả số thực cần sai số cho phép. Trong bài học có thể dùng double; với hệ thống tiền tệ thực tế cần quy tắc làm tròn và kiểu dữ liệu phù hợp.

1.3. Toán tử và chuyển đổi chuỗi - số

Nguồn: S1, trang 6-8.

```java
int a = 7, b = 2;
System.out.println(a / b);          // 3
System.out.println((double) a / b); // 3.5
System.out.println(a % b);          // 1
System.out.println("Tong = " + (a + b)); // Tong = 9
System.out.println("Tong = " + a + b);   // Tong = 72
```

Phép / không luôn là chia nguyên: nếu cả hai toán hạng là số nguyên, kết quả nguyên được cắt về phía 0; có toán hạng số thực thì thực hiện chia số thực. Phép % lấy phần dư, thường dùng kiểm tra chẵn lẻ: n % 2 == 0.

Ở hai dòng cuối, + được xử lý từ trái sang phải. Khi đã gặp chuỗi, nó nối chuỗi: "Tong = " + 7 tạo "Tong = 7", nối tiếp 2 thành "Tong = 72". Ngoặc buộc tính tổng trước khi nối.

Nhóm	Toán tử	Ví dụ
So sánh	==, !=, >, <, >=, <=	diem >= 5
Logic	&&, ||, !	tuoi >= 18 && coVe
Gán	=, +=, -=, *=, /=	tong += x tương đương tong = tong + x trong ca đơn giản
Tăng/giảm	++, --	i++ tăng i lên 1

&& và || có đánh giá ngắn mạch. Trong b != 0 && a / b > 2, nếu b bằng 0 thì vế phải không chạy. Đây là cách tránh chia cho 0 trong điều kiện. Dấu = là gán, còn == là so sánh.

```java
int x = 3;
int y = x++; // y = 3, x = 4
int z = ++x; // x = 5, z = 5
int n = Integer.parseInt("123");
double d = Double.parseDouble("8.5");
String s = String.valueOf(n);
String t = Integer.toString(n);
```

x++ lấy giá trị cũ rồi tăng; ++x tăng rồi lấy giá trị mới. Khi chỉ viết riêng i++; trong vòng lặp, cả hai cách đều tăng i lên 1.

parseInt nhận chuỗi biểu diễn số nguyên; "12a" không hợp lệ và gây NumberFormatException. Có thể trim() để bỏ khoảng trắng ở hai đầu trước khi chuyển. int là kiểu nguyên thủy nên không gọi n.toString(); hãy dùng hai cách ở trên.

1.4. Nhập xuất: ví dụ hoàn chỉnh

Nguồn: S1, trang 9-13.

JAVA

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Nhap ten: ");
        String ten = sc.nextLine();
        System.out.print("Nhap tuoi: ");
        int tuoi = Integer.parseInt(sc.nextLine().trim());
        System.out.printf("%s, %d tuoi%n", ten, tuoi);
        System.out.print("Nhap a va b: ");
        int a = sc.nextInt();
        int b = sc.nextInt();
        System.out.println("Tong = " + (a + b));
        if (b != 0) {
            System.out.printf("Thuong = %.2f%n", (double) a / b);
        } else {
            System.out.println("Khong chia duoc cho 0");
        }
    }
}
```

Nếu nhập Dat, 21 rồi 7 2: chương trình in “Dat, 21 tuoi”, “Tong = 9”, “Thuong = 3.50” trong locale dùng dấu chấm thập phân. %.2f định dạng hai chữ số sau dấu thập phân; %d dành cho số nguyên; %s cho chuỗi; %n xuống dòng. Java không dùng %i để in số nguyên.

print không tự xuống dòng; println có xuống dòng; printf dùng chuỗi định dạng. Biến sc là một đối tượng Scanner đọc từ System.in.

Vì sao nextLine đôi khi đọc chuỗi rỗng?

nextInt đọc token số nhưng thường để lại dấu xuống dòng sau số. nextLine tiếp theo đọc phần còn lại của chính dòng đó, có thể là chuỗi rỗng. Cách xử lý: gọi thêm sc.nextLine() sau khi đọc số và trước khi nhập dòng văn bản, hoặc nhất quán đọc tất cả bằng nextLine rồi parse như phần tuổi ở trên.

Cách BufferedReader trong slide

JAVA

```java
// Import java.io.BufferedReader và java.io.InputStreamReader.
// Đặt trong phương thức có throws java.io.IOException.
BufferedReader br = new BufferedReader(
    new InputStreamReader(System.in));
int n = Integer.parseInt(br.readLine().trim());
```

readLine có thể phát sinh IOException nên cần khai báo throws hoặc xử lý bằng try/catch. Với các bài đầu, Scanner dễ sử dụng hơn. Nên dùng chung một bộ đọc đầu vào cho chương trình, truyền nó vào các hàm nhập khi cần.

02 / Điều kiện và vòng lặp

2.1. if, else if và if lồng nhau

Nguồn: S2, trang 1-5.

```java
int n = -3;
if (n > 0) {
    System.out.println("Duong");
} else if (n < 0) {
    System.out.println("Am");
} else {
    System.out.println("Zero");
}
```

Java xét từ trên xuống. Với n = -3, n > 0 sai nên bỏ nhánh đầu; n < 0 đúng nên in “Am” rồi ra khỏi cả chuỗi if/else if/else. Khi một nhánh đã được chọn, các nhánh sau không xét nữa.

Các if độc lập không giống một chuỗi else if. Ví dụ if (n > 0) và if (n > 5) có thể cùng chạy khi n = 10. Dùng else if khi muốn chọn một trong các trường hợp loại trừ nhau.

Hai khối gần giống nhưng kết quả khác

```java
int x = 10;
if (x > 5) {
    if (x == 10) {
        x = x + 1;
    }
    x = x + 2;
}
// x = 13
```

Bước 1: 10 > 5 đúng. Bước 2: 10 == 10 đúng, x thành 11. Bước 3: x = x + 2 nằm ngoài if bên trong nhưng vẫn thuộc if bên ngoài, nên tiếp tục chạy và x thành 13.

Nếu thay câu cuối bằng nhánh else của if (x == 10), kết quả sẽ là 11. Khi nhánh đúng đã chạy thì nhánh else không chạy, kể cả x vừa thay đổi.

Ví dụ năm nhuận

```java
boolean nhuan = nam % 400 == 0
    || (nam % 4 == 0 && nam % 100 != 0);
```

Năm 2000 chia hết cho 400 nên nhuận. Năm 1900 chia hết cho 100 nhưng không cho 400 nên không nhuận. Năm 2024 chia hết cho 4, không cho 100 nên nhuận. Ngoặc làm cấu trúc điều kiện dễ đọc hơn.

Lỗi thường gặp: if (n > 0); có dấu ; tạo một câu lệnh rỗng làm thân if, khiến khối phía sau không còn được điều kiện này kiểm soát.

2.2. switch và lựa chọn nhiều nhánh

Nguồn: S2, trang 6-7; S7, trang 11.

switch phù hợp khi so một giá trị với các trường hợp cố định, chẳng hạn số tháng hoặc lựa chọn menu. Ví dụ dưới dùng cú pháp case truyền thống trong slide.

```java
int chon = 2;
switch (chon) {
    case 1:
        System.out.println("Xem danh sach");
        break;
    case 2:
        System.out.println("Them sinh vien");
        break;
    case 3:
        System.out.println("Xoa sinh vien");
        break;
    default:
        System.out.println("Lua chon khong hop le");
}
```

Bước 1: switch nhận giá trị 2. Bước 2: chuyển tới case 2 và in “Them sinh vien”. Bước 3: break thoát switch, nên case 3 không chạy. default chạy nếu không có case khớp.

Thiếu break thì sao?

```java
int x = 1;
switch (x) {
    case 1: System.out.print("A");
    case 2: System.out.print("B"); break;
    default: System.out.print("C");
}
```

Kết quả là AB. Sau khi vào case 1, Java tiếp tục chạy xuống case 2 vì chưa gặp break. Hiện tượng này gọi là fall-through; đôi khi cố ý dùng để gom nhiều case vào một xử lý chung.

Ví dụ gom nhiều giá trị

```java
switch (thang) {
    case 4: case 6: case 9: case 11:
        System.out.println("30 ngay");
        break;
    case 2:
        System.out.println("28 hoac 29 ngay");
        break;
    default:
        System.out.println("Can kiem tra thang hop le");
}
```

Đây là ví dụ gom nhánh, chưa phải hàm tính số ngày đầy đủ: cần xét các tháng 31 ngày và loại dữ liệu ngoài 1..12. switch không tự kiểm tra miền giá trị đầu vào thay cho người viết chương trình.

Trong Java 17, cách dùng cơ bản này hỗ trợ int và một số kiểu liên quan, String, enum; không dùng double làm biến chọn của switch truyền thống.

2.3. for, while, do...while và for-each

Nguồn: S2, trang 8-14.

```java
int tong = 0;
for (int i = 1; i <= 4; i++) {
    tong += i;
}
System.out.println(tong); // 10
```

for chạy theo thứ tự: khởi tạo i một lần → kiểm tra điều kiện → chạy thân → tăng i → quay lại kiểm tra. Khi i = 5, điều kiện sai nên dừng.

Lần	i	tong trước	tong sau
1	1	0	1
2	2	1	3
3	3	3	6
4	4	6	10

```java
int i = 1;
while (i <= 4) {
    System.out.println(i);
    i++;
}
int j = 11;
do {
    System.out.println(j);
    j++;
} while (j < 10);
```

while kiểm tra trước nên có thể không chạy lần nào. do...while chạy thân trước rồi mới kiểm tra, nên ví dụ vẫn in 11 một lần. Dấu ; cuối while của do...while là bắt buộc.

Chọn for khi biết quy luật đếm; while khi số lần chưa rõ; do...while khi muốn nhập hoặc hiện menu ít nhất một lần. Luôn có thao tác giúp tiến tới điều kiện dừng; quên i++ có thể gây vòng lặp vô hạn.

```java
int[] a = {2, 4, 6};
for (int x : a) {
    System.out.println(x);
}
```

for-each lần lượt gán giá trị từng phần tử vào x. Nó tiện để đọc các phần tử mà không cần chỉ số. Với mảng int, viết x = 0 trong thân không làm các phần tử mảng thành 0. Muốn sửa từng ô, dùng a[i] = 0 với vòng for có chỉ số.

Trong ví dụ slide duyệt i từ 0 đến dưới 100 và chỉ in khi i % 10 == 0, kết quả là 0, 10, 20, ..., 90; không có 100 vì điều kiện i < 100.

2.4. break, continue, return và số nguyên tố

Nguồn: S2, trang 15-16, 26.

Lệnh	Tác dụng	Phạm vi thường dùng
break	Thoát vòng lặp hoặc switch gần nhất	Dừng tìm kiếm khi đã thấy
continue	Bỏ phần còn lại của lượt lặp hiện tại	Bỏ các phần tử không phù hợp
return	Kết thúc cả phương thức	Trả về kết quả ngay

JAVA

```java
static int timSoChan() {
    for (int i = 1; i <= 10; i++) {
        if (i % 2 != 0) continue;
        if (i > 5) return i;
    }
    return -1;
}
```

Với i = 1, 3, 5, continue bỏ qua câu if thứ hai. Với i = 2, 4, i > 5 sai. Đến i = 6, return 6 kết thúc phương thức ngay; không duyệt 7..10. Giá trị -1 được chọn làm dấu hiệu “không tìm thấy” nếu vòng lặp kết thúc mà chưa return.

Trong for, continue vẫn chuyển tới phần cập nhật i++. Trong while, nếu đặt bước tăng sau continue thì bước tăng có thể bị bỏ qua, dẫn đến lặp vô hạn. Java có break/continue kèm nhãn để tác động tới vòng lặp ngoài; người mới nên dùng khi thật cần thiết và giữ luồng điều khiển rõ ràng.

Ví dụ phương thức kiểm tra số nguyên tố

JAVA

```java
static boolean laNguyenTo(int n) {
    if (n < 2) return false;
    for (int i = 2; i <= n / i; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
```

Số nguyên tố phải từ 2 trở lên và không có ước ngoài 1 và chính nó. Nếu n có một cặp ước a × b = n thì ít nhất một ước không lớn hơn căn bậc hai của n. Vì vậy chỉ cần thử tới căn n; i <= n / i tránh tràn số của i * i khi n lớn.

Với 13: thử 2 và 3, đều không chia hết, trả true. Với 9: thử 2 không chia hết; thử 3 chia hết, trả false ngay. Với 1: bị loại trước vòng lặp. Muốn in N số nguyên tố đầu tiên, cần một biến đếm số đã in; điều kiện dừng không phải ứng viên <= N.

03 / Mảng và chuỗi

3.1. Mảng một chiều và mảng hai chiều

Nguồn: S2, trang 17-24, 29.

```java
int[] a = {3, 8, 2, 5};
int tong = 0;
int max = a[0];
for (int i = 0; i < a.length; i++) {
    tong += a[i];
    if (a[i] > max) max = a[i];
}
System.out.println(tong); // 18
System.out.println(max);  // 8
```

Mảng có số ô cố định sau khi tạo. Chỉ số hợp lệ là 0..length - 1. a.length là thuộc tính, không viết a.length(). Với mảng trên, a[0] = 3 và a[3] = 5; a[4] vượt phạm vi và gây ArrayIndexOutOfBoundsException.

Khởi tạo max bằng phần tử đầu giúp xử lý cả mảng toàn số âm. Nếu dùng max = 0 với {-5, -2}, kết quả sai là 0. Trước khi truy cập a[0], phải chắc chắn mảng không rỗng. new int[4] tạo 4 ô bằng 0; new String[4] tạo 4 ô tham chiếu bằng null.

Mảng hai chiều là mảng chứa các mảng

```java
int[][] b = {{1, 2, 3}, {4, 5}};
for (int i = 0; i < b.length; i++) {
    for (int j = 0; j < b[i].length; j++) {
        System.out.print(b[i][j] + " ");
    }
    System.out.println();
}
```

Vòng ngoài chọn dòng. Vòng trong duyệt từng cột của dòng hiện tại. Kết quả dòng đầu là 1 2 3; dòng sau là 4 5. b.length = 2, b[0].length = 3 và b[1].length = 2. Java cho phép số cột giữa các dòng khác nhau, nên dùng b[i].length thay vì mặc định tất cả đều có cùng số cột.

Tìm kiếm tuyến tính

Duyệt từ i = 0 đến cuối, so a[i] với x. Nếu bằng thì trả i; duyệt hết chưa thấy thì trả -1. Tìm trong {3, 8, 2, 5} với x = 2 trả chỉ số 2, tức phần tử thứ 3. Phải phân biệt “chỉ số” và “thứ tự phần tử” khi đọc đề.

3.2. Gán mảng, sao chép và thay đổi độ dài

Nguồn: S2, trang 25, 29; S7, trang 3, 7-8.

```java
int[] a = {1, 2, 3};
int[] b = a;
b[0] = 99;
System.out.println(a[0]); // 99
```

b = a chỉ sao chép tham chiếu. Hai biến cùng trỏ tới một mảng nên sửa qua b cũng thấy qua a. Nếu muốn có vùng chứa độc lập, cần tạo mảng mới và sao chép phần tử.

```java
int[] c = java.util.Arrays.copyOf(a, a.length);
c[0] = 7;
System.out.println(a[0]); // v□n 99

char[] nguon = {'d','e','c','a','f','f','e',
                'i','n','a','t','e','d'};
char[] dich = new char[7];
System.arraycopy(nguon, 2, dich, 0, 7);
System.out.println(dich); // caffein
```

arraycopy nhận lần lượt: mảng nguồn, vị trí bắt đầu nguồn, mảng đích, vị trí bắt đầu đích, số phần tử. Nó lấy 7 ký tự từ chỉ số 2 đến 8: c, a, f, f, e, i, n. Đây là “caffein”, không phải toàn bộ chuỗi nguồn.

Thêm cuối và xóa một vị trí

```java
int[] ds = {10, 20, 30};
ds = java.util.Arrays.copyOf(ds, ds.length + 1);
ds[ds.length - 1] = 40; // {10, 20, 30, 40}
int k = 1; // xóa ch□ s□ 1, tức giá trị 20
for (int j = k; j < ds.length - 1; j++) {
    ds[j] = ds[j + 1];
}
ds = java.util.Arrays.copyOf(ds, ds.length - 1);
// {10, 30, 40}
```

copyOf tạo mảng mới, không kéo dài chính đối tượng mảng cũ. Xóa gồm hai bước: dịch trái để lấp chỗ trống rồi thu ngắn mảng. Sau dịch, mảng tạm là {10, 30, 40, 40}; thu ngắn bỏ ô cuối bị lặp.

Phải kiểm tra 0 <= k < ds.length. Với mảng đối tượng, copyOf chỉ sao chép các tham chiếu, không tự tạo các đối tượng độc lập; phần constructor sao chép sẽ giải thích sự khác biệt này.

3.3. Chuỗi và các thao tác thường dùng

Nguồn: S2, trang 30. Phần API và ví dụ dưới bổ sung cho yêu cầu bài tập.

String là đối tượng biểu diễn chuỗi bất biến: các thao tác như trim và replaceAll trả chuỗi kết quả, không sửa trực tiếp nội dung chuỗi cũ. Muốn giữ thay đổi, phải nhận giá trị trả về.

```java
String s = "  Phan   Tien Dat  ";
s = s.trim().replaceAll("\\s+", " ");
System.out.println(s);          // Phan Tien Dat
System.out.println(s.length()); // 13
String[] tu = s.isEmpty() ? new String[0] : s.split(" ");
System.out.println(tu.length);  // 3
for (String t : tu) System.out.println(t);
```

trim bỏ khoảng trắng thông dụng ở hai đầu. Mẫu biểu thức chính quy \s+ khớp một hoặc nhiều ký tự trắng; trong mã Java phải viết "\\s+" để biểu diễn mẫu đó. replaceAll thay mỗi cụm bằng một dấu cách. Xử lý chuỗi rỗng trước khi split giúp đếm số từ rỗng là 0.

Cắt chuỗi theo chỉ số

```java
String s = "Java OOP";
int k = 4;
String trai = s.substring(0, k);          // Java
String phai = s.substring(s.length()-3);  // OOP
String giua = s.substring(5, 8);          // OOP
```

substring(start, end) lấy từ start đến trước end, không gồm end. Vì vậy substring(5, 8) lấy chỉ số 5, 6, 7. Muốn lấy n ký tự từ k, dùng substring(k, k + n), sau khi kiểm tra k >= 0, n >= 0 và k + n <= s.length().

So sánh nội dung

```java
String a = new String("Java");
String b = new String("Java");
System.out.println(a == b);      // false
System.out.println(a.equals(b)); // true
```

== trên tham chiếu kiểm tra có cùng đối tượng hay không; equals của String kiểm tra nội dung. Khi tìm mã sinh viên, dùng equals. Có thể dùng "A001".equals(ma) để tránh lỗi nếu ma là null.

Bổ sung: length và substring dùng đơn vị UTF-16. Với bài tập chữ thông thường cách trên phù hợp; chuỗi có emoji hoặc dấu tổ hợp cần xử lý Unicode kỹ hơn nếu đề yêu cầu đếm ký tự hiển thị.

04 / Tư duy hướng đối tượng

4.1. Từ một dãy lệnh đến các lớp có trách nhiệm

Nguồn: S3, toàn bộ 17 trang.

Lập trình tuyến tính mô tả công việc như một dãy lệnh. Cách này hợp bài ngắn: nhập a, nhập b, tính tổng, in kết quả. Khi nhiều công việc lặp lại, ta tách thành các hàm trong lập trình cấu trúc/thủ tục.

```java
static int tong(int a, int b) {
    return a + b;
}
```

Hàm trên tái sử dụng được ở nhiều nơi. Vì vậy không nên hiểu rằng lập trình thủ tục "không tái sử dụng được mã". Vấn đề thường gặp là dữ liệu và các thao tác liên quan nằm rải rác; khi thay đổi mô hình dữ liệu, nhiều nơi có thể cần sửa. OOP giúp tổ chức trách nhiệm, nhưng thiết kế kém vẫn có thể làm mã khó bảo trì.

Phân tích bài toán quản lý sinh viên

Bước 1 - Xác định thực thể: sinh viên và danh sách sinh viên.

Bước 2 - Xác định dữ liệu của một sinh viên: mã, họ tên, điểm các môn. Mã nên dùng String khi có chữ hoặc số 0 đầu; không xem mã định danh là đại lượng để tính toán.

Bước 3 - Xác định hành vi: tính điểm trung bình thuộc SinhVien vì nó dùng điểm của chính sinh viên đó. Tìm một người theo mã thuộc DanhSachSinhVien vì phải duyệt nhiều người.

Bước 4 - Xác định quy tắc: điểm nằm trong [0, 10], mã không rỗng, mã trong danh sách không trùng. Đặt kiểm tra gần nơi thay đổi dữ liệu để không phải nhớ kiểm tra ở mọi màn hình.

Khái niệm	Hiểu qua ví dụ
Lớp	SinhVien là khuôn mô tả dữ liệu và hành vi
Đối tượng	Một sinh viên cụ thể có mã A001
Đóng gói	Giữ điểm private, chỉ nhận giá trị hợp lệ qua phương thức
Kế thừa	SinhVien là một dạng Nguoi
Đa hình	Cùng gọi xuat(), mỗi loại người xuất thông tin riêng
Trừu tượng hóa	Chỉ chọn chi tiết liên quan tới bài toán quản lý

Đối tượng phần mềm còn có thể biểu diễn khái niệm như đơn hàng, phiên đăng nhập, bài thi; không bắt buộc là vật thể chạm được. Lớp là kiểu/khuôn; instance là một đối tượng cụ thể tạo từ lớp đó.

05 / Lớp và đối tượng

5.1. Một lớp hoàn chỉnh và tính đóng gói

Nguồn: S4, trang 3-10, 28-32.

JAVA

```java
class HinhChuNhat {
    private double dai;
    private double rong;

    public HinhChuNhat(double dai, double rong) {
        setDai(dai);
        setRong(rong);
    }
    public double getDai() { return dai; }
    public double getRong() { return rong; }
    public void setDai(double dai) {
        if (!Double.isFinite(dai) || dai <= 0)
            throw new IllegalArgumentException("Dai phai duong");
        this.dai = dai;
    }
    public void setRong(double rong) {
        if (!Double.isFinite(rong) || rong <= 0)
            throw new IllegalArgumentException("Rong phai duong");
        this.rong = rong;
    }
    public double dienTich() { return dai * rong; }
    public double chuVi() { return 2 * (dai + rong); }
}
```

private ngăn mã bên ngoài truy cập trực tiếp dai và rong. Setter không chỉ để “gán hộ”; nó giữ quy tắc cạnh phải là số hữu hạn dương. throw ở đây báo dữ liệu không hợp lệ và không tiếp tục gán. Getter cho đọc giá trị mà không tự cấp quyền sửa.

JAVA

```java
HinhChuNhat h = new HinhChuNhat(5, 3);
System.out.println(h.dienTich()); // 15.0
h.setDai(7);
System.out.println(h.chuVi());    // 20.0
```

Bước 1: new cấp phát đối tượng và thực hiện constructor với 5, 3. Bước 2: constructor kiểm tra rồi lưu các cạnh. Bước 3: h giữ tham chiếu tới đối tượng đó. Bước 4: h.dienTich() dùng dữ liệu của chính h. Sau setDai(7), chỉ dai đổi; rong vẫn là 3.

Từ khóa this chỉ đối tượng hiện tại. Trong this.dai = dai, bên trái là thuộc tính; bên phải là tham số có cùng tên. Nếu viết dai = dai, bạn chỉ gán tham số cho chính nó, thuộc tính không thay đổi.

5.2. Phạm vi truy cập và ba loại biến

Nguồn: S4, trang 9-15, 20, 33-35.

Thành viên	Trong lớp	Cùng package	Lớp con khác package	Nơi khác
private	Có	Không	Không	Không
Không ghi modifier	Có	Có	Không	Không
protected	Có	Có	Có, có điều kiện	Không
public	Có	Có	Có	Có

Bảng mô tả thành viên trong mô hình lớp thông thường. protected ở lớp con khác package không có nghĩa là được truy cập qua mọi đối tượng lớp cha bất kỳ; quyền truy cập còn phụ thuộc kiểu biểu thức nhận. Lớp top-level chỉ dùng public hoặc không ghi modifier; private/protected dùng được cho lớp lồng, không cho lớp top-level.

JAVA

```
class SinhVien {
    static int soDaTao = 0; // bi□n lớp, dùng chung
    private String ten;    // bi□n instance, m□i đ□i tượng riêng
    SinhVien(String ten) {
        int doDai = ten.length(); // bi□n cục bộ
        this.ten = ten;
        soDaTao++;
    }
}
```

Tạo hai đối tượng thì có hai ô ten riêng, nhưng cùng dùng một trường soDaTao của lớp. doDai chỉ có phạm vi trong constructor. Sau khi ra khỏi phạm vi, mã bên ngoài không thể gọi doDai.

Trường instance và static có giá trị mặc định: số là 0, boolean là false, tham chiếu là null. Biến cục bộ phải được gán chắc chắn trước khi đọc; Java không tự gán 0 để bạn sử dụng một biến cục bộ chưa khởi tạo.

static không đồng nghĩa với final

static nói thành viên thuộc lớp; final nói biến không được gán lại. static int dem có thể tăng giảm; static final int MAX = 100 là hằng dùng chung. Phương thức static không có this.

JAVA

```
static int binhPhuong(int x) { return x * x; }
```

Có thể gọi TenLop.binhPhuong(4) mà không new. Hàm static vẫn có thể truy cập thành viên instance thông qua một đối tượng cụ thể, ví dụ h.dienTich(); điều nó không thể làm là dùng trực tiếp dai khi không có đối tượng nhận.

Bộ đếm tăng trong constructor là số lần tạo đối tượng, không tự giảm khi đối tượng không còn được dùng; nó không phải số đối tượng hiện còn sống.

5.3. Constructor, this(...) và sao chép đối tượng

Nguồn: S4, trang 16-19, 28-30.

```java
class Diem {
    private int x, y;
    public Diem() { this(0, 0); }
    public Diem(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public Diem(Diem other) { this(other.x, other.y); }
    public void diChuyen(int dx, int dy) {
        x += dx;
        y += dy;
    }
    public String toString() { return "(" + x + ", " + y + ")"; }
}
```

Constructor trùng tên lớp và không có kiểu trả về, kể cả void. Nó có thể public, protected, private hoặc package-private tùy thiết kế; không bắt buộc public. Nếu không khai báo constructor nào, Java tạo constructor mặc định không tham số. Nếu đã tự khai báo một constructor có tham số, Java không tự thêm constructor không tham số.

this(0, 0) gọi constructor khác của cùng lớp. Theo cú pháp Java 17 dùng trong tài liệu, lời gọi this(...) hoặc super(...) phải đứng đầu thân constructor. Chuỗi gọi phải kết thúc, không được gọi vòng qua lại vô hạn.

```java
Diem a = new Diem(2, 3);
Diem b = a;
Diem c = new Diem(a);
b.diChuyen(1, 0);
System.out.println(a); // (3, 3)
System.out.println(c); // (2, 3)
```

Bước 1: a giữ đối tượng (2, 3). Bước 2: b = a tạo thêm một tham chiếu, không tạo điểm mới. Bước 3: constructor sao chép tạo điểm mới c có cùng giá trị tọa độ. Bước 4: sửa b thực chất sửa đối tượng của a, còn c độc lập.

Sao chép nông và sao chép sâu

Nếu lớp chứa thuộc tính đối tượng, chẳng hạn DiaChi, phép this.diaChi = other.diaChi vẫn chia sẻ cùng địa chỉ; đó là sao chép nông. Muốn độc lập phải sao chép tiếp DiaChi và các thành phần thay đổi được khi cần. Java không tự cung cấp “copy constructor” đặc biệt; Diem(Diem other) là constructor do người viết chủ động định nghĩa.

null là không tham chiếu tới đối tượng. Gọi phương thức qua null gây NullPointerException. Constructor sao chép trên giả định other khác null.

5.4. Phương thức, giá trị trả về và truyền tham số

Nguồn: S4, trang 21-27. Hiệu chỉnh trọng tâm: Java luôn truyền theo giá trị.

```java
static int cong(int a, int b) {
    return a + b;
}
static void tang(int x) {
    x++;
}
```

int ở cong là kiểu trả về; a, b là tham số hình thức. Trong cong(2, 3), 2 và 3 là đối số. return đưa kết quả về nơi gọi, vì vậy int k = cong(2, 3) làm k bằng 5. In ra màn hình và trả về là hai việc khác nhau. void không trả giá trị nhưng vẫn có thể dùng return; để thoát sớm.

Nếu int n = 10 rồi gọi tang(n), n vẫn là 10. Hàm nhận bản sao giá trị 10 vào x và chỉ tăng bản sao đó lên 11.

Vì sao truyền đối tượng vẫn sửa được dữ liệu bên trong?

```java
class Hop { int giaTri = 10; }
static void sua(Hop h) {
    h.giaTri = 20;
}
static void ganLai(Hop h) {
    h = new Hop();
    h.giaTri = 99;
}
// Trong main:
Hop a = new Hop();
sua(a);
System.out.println(a.giaTri); // 20
ganLai(a);
System.out.println(a.giaTri); // v□n 20
```

Khi gọi sua(a), giá trị được sao chép là tham chiếu tới Hop. a ở main và h trong hàm là hai biến khác nhau nhưng cùng chỉ một đối tượng. Thay h.giaTri sửa đối tượng chung nên main thấy 20.

Khi gọi ganLai(a), h ban đầu cũng chỉ đối tượng của a. Tuy nhiên h = new Hop() chỉ làm biến h cục bộ chuyển sang đối tượng mới; biến a không đổi. Vì vậy 99 nằm trong đối tượng mới của h, không phải đối tượng a đang giữ.

Cách diễn đạt “truyền đối tượng bằng tham chiếu” trong slide dễ dẫn đến kết luận sai rằng hàm có thể gán lại biến tham chiếu của bên gọi. Cách nhớ chính xác: sao chép giá trị; giá trị ấy có thể là một tham chiếu. Quy tắc này cũng áp dụng với mảng.

06 / Kế thừa

6.1. extends, super và trình tự khởi tạo

Nguồn: S5, trang 1-14.

JAVA

```java
class Nguoi {
    private String hoTen;
    public Nguoi(String hoTen) { this.hoTen = hoTen; }
    public void xuat() { System.out.print(hoTen); }
}
class SinhVien extends Nguoi {
    private String ma;
    public SinhVien(String hoTen, String ma) {
        super(hoTen);
        this.ma = ma;
    }
    @Override
    public void xuat() {
        super.xuat();
        System.out.println(" - " + ma);
    }
}
```

Nguoi là lớp cha (base/superclass), SinhVien là lớp con (derived/subclass). extends thể hiện SinhVien là một loại Nguoi. Lớp con có phần trạng thái của cha và thêm ma; không cần khai báo lại hoTen.

Với new SinhVien("Dat", "A001"), constructor con gọi super("Dat") để phần Nguoi lưu hoTen, rồi con lưu ma. Khi gọi xuat, phương thức con dùng super.xuat() in phần tên và tiếp tục in mã. Kết quả là Dat - A001.

Ba điểm cần phân biệt

super(...) gọi constructor cha; super.xuat() gọi cách cài đặt phương thức ở cha; this.ma chỉ thuộc tính của đối tượng hiện tại. super không tạo thêm một đối tượng cha độc lập bên cạnh đối tượng con.

Constructor không được kế thừa. Nếu constructor con không gọi this(...) hoặc super(...) tường minh, Java 17 chèn super(). Điều đó chỉ hợp lệ khi cha có constructor không tham số truy cập được. Ví dụ Nguoi ở trên chỉ có constructor nhận tên, nên phải gọi super(hoTen).

Lớp con không trực tiếp truy cập trường private của cha; nó làm việc qua phương thức mà cha cho phép. Các thành viên package-private chỉ được truy cập khi phù hợp package. Mọi lớp thông thường, trừ Object, đều có Object trong chuỗi lớp cha.

Java hỗ trợ một lớp cha trực tiếp. Kế thừa nhiều cấp: A → B → C. Kế thừa thứ bậc: B và C cùng kế thừa A. Không viết class C extends A, B; một lớp có thể thực hiện nhiều interface.

6.2. is-a, has-a và ép kiểu đối tượng

Nguồn: S5, trang 15-21; S6, trang 11-14.

is-a diễn đạt “là một loại”: DiemMau là một Diem nên có thể kế thừa. has-a diễn đạt “có/chứa”: DoanThang có hai DiemMau nên khai báo hai thuộc tính kiểu DiemMau, không cho DoanThang kế thừa DiemMau.

Tương tự, Nguoi có NoiSinh; PhongKhach là Phong và có Sofa. Chỉ việc có một trường tham chiếu chưa đủ kết luận là composition trong UML: cần xét quyền sở hữu và vòng đời ở phần sơ đồ lớp.

Upcasting: nhìn đối tượng con qua kiểu cha

JAVA

```java
class DongVat {
    public void keu() { System.out.println("AAA"); }
}
class Meo extends DongVat {
    @Override
    public void keu() { System.out.println("Meo meo"); }
    public void cao() { System.out.println("Cao mong"); }
}
DongVat dv = new Meo(); // đặt câu lệnh này trong main
```

Đối tượng thật vẫn là Meo; biến dv có kiểu khai báo DongVat. Upcasting không biến một con mèo thành một đối tượng DongVat mới và không xóa dữ liệu riêng của nó. dv.keu() gọi cách cài đặt của Meo. Nhưng dv.cao() không biên dịch vì kiểu DongVat không khai báo cao().

Downcasting: lấy lại góc nhìn lớp con

JAVA

```java
if (dv instanceof Meo) {
    Meo m = (Meo) dv;
    m.cao();
}
```

instanceof kiểm tra đối tượng có phù hợp với kiểu Meo hay không. Nếu đúng, ép kiểu để được gọi phương thức riêng. Với dv = new DongVat(), ép thẳng (Meo) dv gây ClassCastException. Ép kiểu không tạo ra các đặc tính còn thiếu của một đối tượng cha.

null instanceof Meo cho false. Khi mục tiêu chỉ là gọi keu(), không cần instanceof: đa hình đã chọn đúng cách cài đặt. Chỉ kiểm tra kiểu khi thực sự cần hành vi đặc thù chưa có trong hợp đồng chung.

Trong bài đoạn thẳng, khoảng cách hai điểm là sqrt((x2-x1)² + (y2-y1)²). Với (0, 0) và (3, 4), khoảng cách là 5. Đó là hành vi của DoanThang dùng dữ liệu từ hai điểm thành phần.

07 / Đa hình và trừu tượng

7.1. Ghi đè và quyết định phương thức lúc chạy

Nguồn: S6, trang 1-10.

JAVA

```java
class DongVat {
    public void keu() { System.out.println("AAA"); }
}
class Meo extends DongVat {
    @Override
    public void keu() { System.out.println("Meo meo"); }
}
class Cho extends DongVat {
    @Override
    public void keu() { System.out.println("Gau gau"); }
}
// Trong main:
DongVat[] ds = {new DongVat(), new Cho(), new Meo()};
for (DongVat x : ds) x.keu();
```

Kết quả theo thứ tự: AAA, Gau gau, Meo meo. Cùng một câu x.keu(), nhưng mỗi đối tượng đáp ứng theo cách cài đặt của lớp thật của nó. Đây là đa hình lúc chạy đối với phương thức instance có thể ghi đè.

Lượt	Kiểu khai báo của x	Lớp thật	Phương thức chạy
1	DongVat	DongVat	DongVat.keu
2	DongVat	Cho	Cho.keu
3	DongVat	Meo	Meo.keu

Kiểu khai báo quyết định thành viên nào được phép gọi khi biên dịch. Lớp thật quyết định bản ghi đè nào chạy. Khi gán x = đối tượng khác, chính biến x đổi nơi tham chiếu; một đối tượng không tự biến đổi lớp của nó.

Quy tắc ghi đè

Tên và danh sách kiểu tham số phải phù hợp với phương thức được ghi đè. Kiểu trả về phải tương thích; với tham chiếu có thể là kiểu con (covariant return). Không được giảm mức truy cập, chẳng hạn từ public xuống private.

@Override giúp trình biên dịch bắt lỗi nếu bạn định ghi đè nhưng viết sai tên hoặc tham số. Nó không phải thứ tạo ra đa hình; nếu chữ ký đúng, phương thức vẫn ghi đè khi thiếu annotation.

Phương thức final không được ghi đè. Phương thức private không được lớp con kế thừa để ghi đè. Phương thức static được che (hiding), không chọn theo cơ chế đa hình instance. Thuộc tính cũng không được chọn động giống phương thức, nên tránh khai báo trường trùng tên ở cha và con.

7.2. Lớp trừu tượng và phương thức trừu tượng

Nguồn: S6, trang 18-24.

JAVA

```java
abstract class HinhHoc {
    public abstract double dienTich();
    public abstract double chuVi();
    public void xuat() {
        System.out.printf("S = %.2f; P = %.2f%n",
                          dienTich(), chuVi());
    }
}
class HinhTron extends HinhHoc {
    private double r;
    public HinhTron(double r) {
        if (!Double.isFinite(r) || r <= 0)
            throw new IllegalArgumentException("r phai duong");
        this.r = r;
    }
    @Override public double dienTich() { return Math.PI*r*r; }
    @Override public double chuVi() { return 2*Math.PI*r; }
}
```

Phương thức abstract chỉ khai báo hợp đồng, kết thúc bằng ; và không có thân. Lớp có phương thức abstract phải là abstract. Chiều ngược lại không bắt buộc: một lớp abstract có thể không có phương thức abstract nào.

Không được new HinhHoc(). Nhưng HinhHoc h = new HinhTron(2) hợp lệ: biến kiểu trừu tượng đang tham chiếu đến một đối tượng cụ thể.

Theo dõi lời gọi h.xuat()

Bước 1: HinhTron không ghi đè xuat nên dùng thân xuat của HinhHoc.

Bước 2: bên trong xuat gọi dienTich(); đối tượng hiện tại là HinhTron nên dùng công thức πr^2 của HinhTron.

Bước 3: chuVi() cũng dùng phiên bản HinhTron. Với r = 2, diện tích và chu vi đều xấp xỉ 12.57 sau làm tròn hai chữ số.

Lớp abstract có thể chứa trường, constructor và phương thức đã cài đặt. Constructor của nó vẫn được gọi khi tạo lớp con để khởi tạo phần trạng thái chung.

Nếu một lớp con cụ thể chưa cài đặt đủ các phương thức abstract được thừa hưởng, nó không biên dịch; hoặc phải tiếp tục khai báo lớp con đó là abstract. Với bài hình học, có thể khai báo thêm nhap(Scanner sc) theo yêu cầu; kiểm tra kích thước hợp lệ trước khi lưu.

7.3. Interface: một khả năng mà nhiều lớp có thể có

Nguồn: S8, trang 18; S9, trang 3-4. Diễn giải bổ sung về cú pháp.

JAVA

```java
interface Resizable {
    void resize(int percent);
}
class Circle {
    protected double radius;
    public Circle(double radius) { this.radius = radius; }
    public double area() { return Math.PI * radius * radius; }
}
class ResizableCircle extends Circle implements Resizable {
    public ResizableCircle(double radius) { super(radius); }
    @Override
    public void resize(int percent) {
        if (percent <= 0)
            throw new IllegalArgumentException("percent > 0");
        radius *= percent / 100.0;
    }
}
```

Resizable diễn đạt “có khả năng thay đổi kích thước”. Một lớp dùng extends cho lớp cha và implements cho interface. Phương thức resize trong interface này ngầm là public abstract, nên phương thức thực hiện phải public.

Mẫu Circle ở đây tối giản để giải thích interface; ví dụ tích hợp phần 10 sẽ gắn thêm Shape và chu vi. Đầu vào bán kính của ví dụ tối giản giả định là số hữu hạn dương.

JAVA

```java
ResizableCircle c = new ResizableCircle(2);
Resizable r = c;
r.resize(150);
System.out.println(c.area()); // 9 * PI, kho□ng 28.2743
```

Ban đầu radius = 2. Tỷ lệ = 150 / 100.0 = 1.5. Bán kính mới = 3, diện tích mới = 9π. Nếu dùng percent / 100 với hai int, 150 / 100 chỉ là 1 và kết quả sai.

Abstract class	Interface
Chia sẻ trạng thái và mã giữa các lớp có liên hệ	Mô tả hợp đồng/khả năng chung
Có constructor và trường instance	Không có constructor; trường khai báo là hằng static final
Một lớp chỉ extends một lớp trực tiếp	Một lớp có thể implements nhiều interface
Có phương thức abstract và phương thức có thân	Ngoài abstract, Java 17 còn cho default, static, private

Không khởi tạo trực tiếp interface. Có thể dùng biến hoặc mảng kiểu interface để quản lý nhiều đối tượng cùng đáp ứng hợp đồng, dù chúng thuộc các cây kế thừa khác nhau.

7.4. final và lớp lồng nhau

Nguồn: S6, trang 15-17; S1, trang 5.

Vị trí final	Ý nghĩa
Biến	Không gán lại sau khi đã gán
Phương thức	Lớp con không được ghi đè
Lớp	Không được tạo lớp con kế thừa

```java
final int[] a = {1, 2};
a[0] = 9; // hợp lệ
// a = new int[3]; // không hợp lệ
```

final áp dụng lên biến tham chiếu a, không làm toàn bộ đối tượng mảng bất biến. Có thể thay đổi nội dung ô nhưng không cho a trỏ sang mảng khác. Đây là điểm khác với bất biến của String.

Inner class và static nested class

```java
class BenNgoai {
    private int x = 10;
    class BenTrong {
        int doc() { return x; }
    }
    static class TienIch {
        int binhPhuong(int n) { return n * n; }
    }
}
// Trong main:
BenNgoai ngoai = new BenNgoai();
BenNgoai.BenTrong trong = ngoai.new BenTrong();
System.out.println(trong.doc()); // 10
BenNgoai.TienIch t = new BenNgoai.TienIch();
System.out.println(t.binhPhuong(3)); // 9
```

BenTrong không static nên gắn với một đối tượng BenNgoai cụ thể. Khi doc() đọc x, nó đọc x của đối tượng ngoai gắn với nó. Lớp lồng có thể truy cập thành viên private của lớp ngoài trong phạm vi cho phép của ngôn ngữ.

TienIch là static nested class, không tự gắn với đối tượng BenNgoai. Vì vậy nó không thể trực tiếp dùng x nếu chưa có tham chiếu tới một BenNgoai. Có thể tạo nó bằng new BenNgoai.TienIch() mà không tạo ngoai trước.

Lớp lồng hữu ích khi một kiểu chỉ hỗ trợ nội bộ cho kiểu ngoài, chẳng hạn phần tử Node của một cấu trúc dữ liệu. Không nên lồng mọi lớp: các thực thể dùng độc lập như SinhVien và DanhSachSinhVien thường dễ hiểu hơn khi tách riêng.

08 / Nạp chồng và lớp danh sách

8.1. Overloading khác overriding thế nào?

Nguồn: S7, trang 2; S6, trang 6.

```java
static int tong(int a, int b) { return a + b; }
static double tong(double a, double b) { return a + b; }
static int tong(int a, int b, int c) { return a + b + c; }
```

Ba phương thức cùng tên nhưng khác danh sách tham số là nạp chồng (overloading). Có thể khác số tham số, kiểu tham số hoặc thứ tự kiểu. Không thể chỉ thay kiểu trả về để tạo overload: int f(int x) và double f(int x) xung đột.

Tiêu chí	Overloading	Overriding
Mục tiêu	Cùng tên, nhiều cách nhận đối số	Lớp con đổi cách thực hiện hợp đồng cha
Danh sách tham số	Phải khác	Phải phù hợp với phương thức cha
Cần kế thừa?	Không	Có quan hệ thừa hưởng/hiện thực hợp đồng
Quyết định chính	Lúc biên dịch theo kiểu đối số	Lúc chạy theo đối tượng, với phương thức instance

Gọi tong(2, 3) chọn bản int; tong(2.5, 3.0) chọn bản double; tong(1, 2, 3) chọn bản ba tham số. Đây không phải việc Java thử chạy từng hàm rồi xem hàm nào đúng.

Ví dụ dễ nhầm về kiểu khai báo

```java
static void xem(Object x) { System.out.println("Object"); }
static void xem(String x) { System.out.println("String"); }
// Trong main:
Object a = "Java";
xem(a);          // Object
xem((String) a); // String
```

Dù đối tượng thật là String, biểu thức a có kiểu khai báo Object nên overload xem(Object) được chọn. Ở lời gọi thứ hai, biểu thức ép kiểu có kiểu String nên chọn xem(String).

Trong lớp danh sách, có thể viết them() để nhập từ bàn phím và them(SinhVien x) để nhận đối tượng đã có; hoặc xoa() để hỏi mã rồi gọi xoa(String ma). Nên dồn logic chính vào phương thức nhận tham số để tránh viết hai lần cùng một thuật toán.

8.2. Thiết kế SinhVien và DanhSachSinhVien

Nguồn: S7, trang 3-6, 9-11.

```java
class SinhVien {
    private String ma, ten, sdt;
    public SinhVien(String ma, String ten, String sdt) {
        if (ma == null || ma.trim().isEmpty())
            throw new IllegalArgumentException("Ma rong");
        this.ma = ma;
        this.ten = ten;
        this.sdt = sdt;
    }
    public SinhVien(SinhVien x) { this(x.ma, x.ten, x.sdt); }
    public String getMa() { return ma; }
    public String getTen() { return ten; }
    public String getSdt() { return sdt; }
    public void setTen(String ten) { this.ten = ten; }
    public void setSdt(String sdt) { this.sdt = sdt; }
    public String toString() { return ma + " - " + ten; }
}
```

Ví dụ giữ mã ổn định sau khi tạo để việc sửa tên không làm mất tính duy nhất của mã. Số điện thoại là String vì không dùng để tính toán và có thể có số 0 đầu. Copy constructor sao chép các trường String là đủ cho dữ liệu này vì String bất biến.

Bất biến của danh sách

```java
class DanhSachSinhVien {
    private SinhVien[] ds = new SinhVien[0];
    private int n = 0;
    // Các phương thức  hai trang sau đặt trong lớp này.
}
```

Chọn quy ước n == ds.length và mọi phần tử 0..n-1 khác null. Mỗi lần thêm/xóa phải cập nhật cả mảng và n. Đây là cách bám slide; trong thiết kế khác có thể bỏ n và dùng ds.length để tránh dữ liệu trùng lặp.

new SinhVien[3] chỉ tạo ba ô tham chiếu null, chưa tạo ba sinh viên. Muốn nhập vào từng người, phải gán mỗi ô bằng một đối tượng trước khi gọi phương thức của nó. Duyệt ô null rồi gọi xuat() sẽ gây NullPointerException.

Phân trách nhiệm: SinhVien giữ và xử lý một sinh viên; DanhSachSinhVien quản lý nhiều người; lớp menu nhận lựa chọn và gọi phương thức danh sách. Không đưa toàn bộ thêm/sửa/xóa vào main vì sẽ khó đọc và khó tái sử dụng.

8.3. Tìm kiếm, thêm và sửa: theo dõi từng bước

Nguồn: S7, trang 7, 9-11. Đặt các phương thức dưới trong DanhSachSinhVien.

```java
public int timViTri(String ma) {
    for (int i = 0; i < n; i++) {
        if (ds[i].getMa().equals(ma)) return i;
    }
    return -1;
}
public boolean them(SinhVien x) {
    if (x == null || timViTri(x.getMa()) != -1) return false;
    ds = java.util.Arrays.copyOf(ds, n + 1);
    ds[n] = new SinhVien(x);
    n++;
    return true;
}
public boolean sua(String ma, String tenMoi, String sdtMoi) {
    int i = timViTri(ma);
    if (i == -1) return false;
    ds[i].setTen(tenMoi);
    ds[i].setSdt(sdtMoi);
    return true;
}
```

Giả sử ds có A001 và A002, n = 2. Thêm A003: tìm không thấy mã, copyOf tạo mảng dài 3 chứa hai tham chiếu cũ và một ô null. ds[2] nhận đối tượng sao chép của x, sau đó n tăng thành 3. Không tăng n trước rồi ghi ds[n], vì sẽ ghi ra ngoài mảng.

Nếu thêm lại A002, timViTri trả 1 nên hàm trả false trước khi thay đổi dữ liệu. Copy constructor ở them giúp người gọi sửa tên của đối tượng x sau đó mà không vô tình đổi bản được lưu trong danh sách.

Sửa A002: tìm ra chỉ số 1, đổi ten và sdt tại đúng đối tượng đó; n và chiều dài mảng không đổi. Nếu không thấy, trả false. Không được dùng chỉ số -1 để truy cập ds.

Trả đối tượng hay bản sao?

```java
public SinhVien timKiem(String ma) {
    int i = timViTri(ma);
    return i == -1 ? null : new SinhVien(ds[i]);
}
```

Bản này trả bản sao để bên gọi không sửa trực tiếp dữ liệu nội bộ. Nếu trả ds[i], bên gọi nhận cùng đối tượng; đó có thể là lựa chọn thiết kế hợp lệ nhưng cần hiểu hậu quả. Trong ví dụ này, muốn lưu thay đổi phải gọi sua(...), không chỉ sửa bản sao tìm được.

Tìm tuyến tính và thêm bằng copyOf đều có chi phí O(n). Với bài học mảng, cách này đơn giản và phù hợp; khi quy mô lớn có thể chọn cấu trúc khác.

8.4. Xóa, xuất và nối vào menu

Nguồn: S7, trang 8, 10-13.

```java
public boolean xoa(String ma) {
    int i = timViTri(ma);
    if (i == -1) return false;
    for (int j = i; j < n - 1; j++) {
        ds[j] = ds[j + 1];
    }
    ds = java.util.Arrays.copyOf(ds, n - 1);
    n--;
    return true;
}
public void xuat() {
    for (SinhVien sv : ds) System.out.println(sv);
}
```

Với [A001, A002, A003], xóa A002 tìm được i = 1. Vòng lặp gán ds[1] = ds[2], mảng tạm thành [A001, A003, A003]. copyOf giữ hai ô đầu, rồi n giảm từ 3 xuống 2. Nếu xóa phần tử cuối, vòng dịch không chạy và chỉ cần thu ngắn mảng.

Kịch bản tự kiểm tra

```java
DanhSachSinhVien ds = new DanhSachSinhVien();
ds.them(new SinhVien("A001", "Dat", "0900000001"));
ds.them(new SinhVien("A002", "Dang", "0900000002"));
System.out.println(ds.them(
    new SinhVien("A001", "Duy", "0900000003"))); // false
ds.sua("A002", "Hai Dang", "0900000002");
ds.xoa("A001");
ds.xuat(); // A002 - Hai Dang
```

Các đoạn ở trang 8.2-8.4 ghép thành hai lớp và một hàm main kiểm tra. Ngoài ca trên, hãy kiểm tra xóa trong danh sách rỗng, xóa mã không tồn tại, xóa người duy nhất, sửa mã không có và tìm kiếm sau xóa.

Menu chỉ điều phối

Dùng do...while để hiện menu ít nhất một lần, đọc lựa chọn rồi switch: 1 xem; 2 thêm; 3 xóa; 4 sửa; 5 tìm; 6 thống kê; 7 thoát. Mỗi case gọi phương thức tương ứng, không lặp lại thuật toán của lớp danh sách.

Với bài văn phòng phẩm hoặc cán bộ, mảng đổi thành VanPhongPham[] hoặc CanBo[]. Khi thêm, menu hỏi loại, tạo lớp con tương ứng rồi lưu vào mảng cha. Khi xuất, chỉ gọi xuat() trên mỗi phần tử: đa hình tự chọn phiên bản của Tap/But hoặc CongNhan/KySu/NhanVien. Quy tắc mã không trùng và xử lý danh sách rỗng vẫn giữ nguyên.

09 / Sơ đồ lớp UML

9.1. Đọc một lớp từ hình vẽ

Nguồn: S8, trang 1-8, 18-19.

Sơ đồ lớp mô tả cấu trúc tĩnh: các lớp, thuộc tính, phương thức và quan hệ. Nó không mô tả thứ tự từng câu lệnh chạy. Một ô lớp thường có ba ngăn: tên lớp, thuộc tính và phương thức.

Ký hiệu	Ý nghĩa	Ví dụ UML
+	public	+ getMa(): String
-	private	- ma: String
#	protected	# radius: double
~	package-private	~ kiemTra(): boolean
Gạch chân	static	Thuộc tính dùng chung của lớp
Chữ nghiêng hoặc {abstract}	Trừu tượng	HinhHoc, dienTich()
«interface»	Interface	Resizable

Chuyển từ UML sang Java

Thuộc tính “- diem: double = 0” nghĩa là private double diem = 0;. Phương thức “+ diChuyen(dx: int, dy: int): void” nghĩa là public void diChuyen(int dx, int dy). UML đặt tên trước kiểu; Java thường đặt kiểu trước tên.

Trong sơ đồ mức thiết kế, nên dùng đúng tên lớp và kiểu sẽ cài đặt. Sơ đồ phân tích có thể bỏ bớt getter/setter hoặc chi tiết cài đặt; cần nhất quán với mục đích bản vẽ và yêu cầu giảng viên.

Ví dụ lớp SinhVien

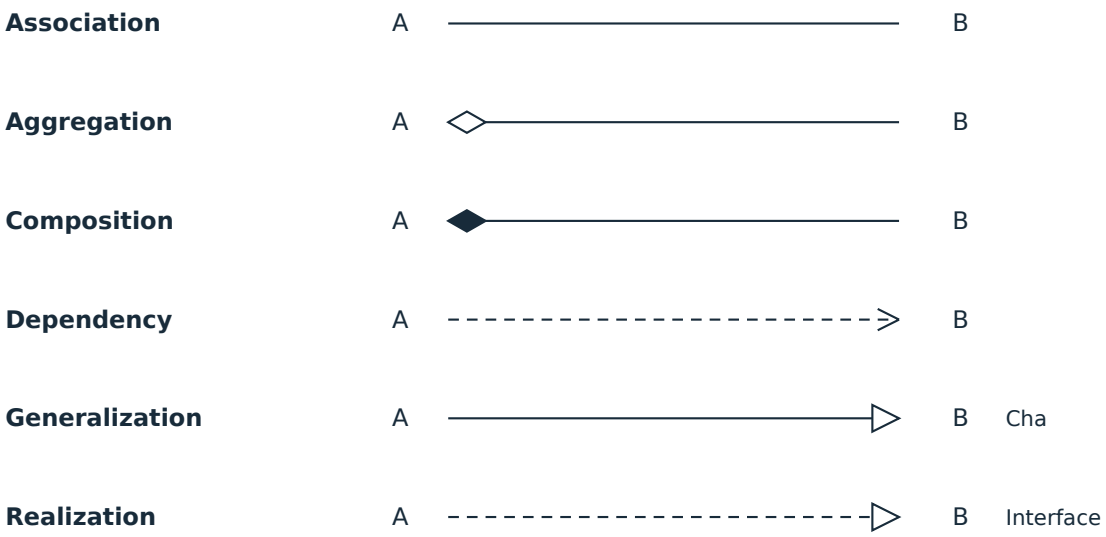


Trong hình, ma và ten được giữ private, còn getMa và xuat là public. Nếu có một trường static như soDaTao thì gạch chân dòng ấy. Constructor có thể ghi cùng tên lớp và các tham số, thường không ghi kiểu trả về.

Phân biệt với ERD: class diagram có phương thức và quan hệ OOP như kế thừa, hiện thực interface. ERD tập trung vào cấu trúc dữ liệu; hai sơ đồ không thay thế hoàn toàn cho nhau.

9.2. Quan hệ và hướng của ký hiệu

Nguồn: S8, trang 9-18.



Đọc đúng ý nghĩa

Association: liên kết cấu trúc giữa các đối tượng. Ví dụ SinhVien đăng ký HocPhan. Nét liền có thể kèm mũi tên điều hướng nếu muốn thể hiện bên nào biết bên nào; không nên xem mọi mũi tên là kế thừa.

Aggregation: quan hệ toàn thể - bộ phận với hình thoi rỗng ở đầu toàn thể. Bộ phận có thể tồn tại độc lập theo mô hình, chẳng hạn đội có thành viên. Không cần dùng kết tập cho mọi thuộc tính mảng; association thường đã đủ nếu không có ý nghĩa toàn thể - bộ phận rõ ràng.

Composition: hình thoi đặc ở đầu chủ sở hữu, diễn đạt sở hữu mạnh. Bộ phận thuộc tối đa một toàn thể tại một thời điểm trong quan hệ hợp thành; vòng đời được mô hình hóa gắn với toàn thể. Ví dụ DonHang có DongDonHang. Đây là ngữ nghĩa thiết kế, không phải một từ khóa Java tự động xóa đối tượng khi chủ bị xóa.

Dependency: nét đứt, mũi tên thường đi từ lớp sử dụng tới lớp được dùng. Ví dụ BaoCao có phương thức in(SinhVien sv) dùng SinhVien trong xử lý.

Generalization: nét liền, tam giác rỗng chỉ về lớp cha. SinhVien kế thừa Nguoi: tam giác nằm phía Nguoi.

Realization: nét đứt, tam giác rỗng chỉ về interface. ResizableCircle thực hiện Resizable: tam giác nằm phía Resizable. Đừng nhầm với dependency chỉ có đầu mũi tên thường.

9.3. Bội số và một mô hình hoàn chỉnh

Nguồn: S8, trang 10-13; liên hệ S7.

Bội số	Cách hiểu
1	Đúng một
0..1	Không có hoặc một
* hoặc 0..*	Từ không đến nhiều
1..*	Ít nhất một
2..4	Từ hai đến bốn

Bội số đặt ở đầu lớp nào thì cho biết số đối tượng của lớp đó có thể liên kết với một đối tượng ở đầu đối diện. Không đọc nó theo thứ tự trái sang phải một cách máy móc.



Bên Professor ghi 0..1: mỗi CourseOffering có thể chưa có giảng viên hoặc có một giảng viên. Bên CourseOffering ghi 0..*: mỗi Professor có thể dạy không lớp học phần nào hoặc nhiều lớp học phần. Đây chính là cách đọc ví dụ trong slide.

Áp dụng quản lý sinh viên

DanhSachSinhVien có thể chứa 0..* SinhVien. Nếu sinh viên vẫn tồn tại độc lập với một danh sách cụ thể, association hoặc aggregation phù hợp hơn composition. Một mảng Java không tự quyết định quyền sở hữu nghiệp vụ.

Các bước vẽ: (1) liệt kê lớp; (2) chọn thuộc tính; (3) gán trách nhiệm thành phương thức; (4) xét is-a để vẽ kế thừa; (5) xét liên kết giữa đối tượng và quyền sở hữu; (6) thêm bội số ở hai đầu; (7) kiểm tra sơ đồ khớp với mã.

Bẫy thường gặp

Một SinhVien có NoiSinh không có nghĩa SinhVien là NoiSinh. Mũi tên kế thừa không được đảo từ cha xuống con. Một lớp Java không extends hai lớp, dù UML tổng quát có thể biểu diễn đa kế thừa cho ngôn ngữ khác. Interface dùng implements; lớp con kế thừa lớp dùng extends.

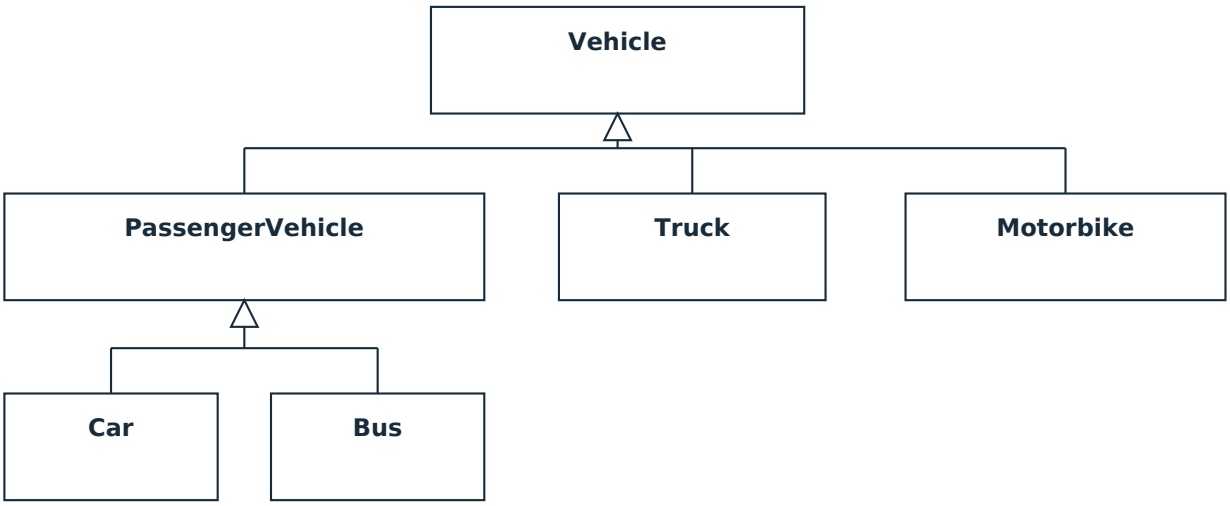
Nếu đề không quy định bội số hoặc vòng đời, phải ghi giả định thay vì coi lựa chọn thiết kế của mình là dữ kiện đã cho. Không đặt composition chỉ vì hai lớp “có liên quan chặt chẽ”.

10 / Vận dụng từ bài tập bổ sung

10.1. Phương tiện: chọn đúng lớp cha chung

Nguồn: S9, trang 1-2.

Đề có Car, Truck, Bus, Motorbike, đều có reg_num và brand. Car và Bus cùng có max_pass. Thiết kế tốt gom dữ liệu chung vào Vehicle, sau đó gom xe chở khách vào PassengerVehicle.



Vehicle giữ số đăng ký và hãng; PassengerVehicle giữ sức chở hành khách; Bus thêm number (mã tuyến); Truck thêm max_load; Motorbike thêm type. Nếu không cần tạo một "xe chung chung", có thể khai báo Vehicle và PassengerVehicle là abstract.

Cách tận dụng super và đa hình

Trong Bus.input(sc): gọi super.input(sc) để nhập thông tin chung và max_pass rồi nhập number. Trong Bus.print(): gọi super.print() rồi in number. Constructor Bus cũng gọi constructor PassengerVehicle để tránh lặp việc gán thuộc tính chung.

```java
// Giả sử các lớp cụ thể đã cài đặt constructor không tham số,
// input(Scanner sc) và print() theo cùng một hợp đồng.
Vehicle[] ds = new Vehicle[2];
ds[0] = new Bus();
ds[1] = new Truck();
for (Vehicle v : ds) v.input(sc);
for (Vehicle v : ds) v.print();
```

Đoạn trên là khung sử dụng, không phải toàn bộ chương trình phương tiện. Khi phần tử là Bus, input/print chạy theo Bus; khi là Truck thì theo Truck. Không cần hai mảng riêng chỉ để in các xe.

Ví dụ một Bus có số đăng ký "51B-001", hãng X, max_pass = 40, tuyến = 8; một Truck có số đăng ký "51C-002", hãng Y, max_load = 5000. Cả hai cùng được quản lý dưới kiểu Vehicle nhưng không ép Truck có thuộc tính max_pass không phù hợp.

Quy tắc cần kiểm tra khi cài đặt: sức chở và tải trọng không âm, số đăng ký không rỗng, thống nhất đơn vị tải trọng là kg theo đề. Đa hình giải quyết hành vi chung; cây kế thừa giải quyết cấu trúc chung.

10.2. Shape và Resizable: hai quan hệ khác nhau

Nguồn: S9, trang 3-4.

Shape mô tả hình có chu vi và diện tích. Circle và Square kế thừa Shape. Resizable mô tả khả năng thay đổi kích thước; ResizableCircle kế thừa Circle và implements Resizable, tương tự với ResizableSquare.

JAVA

```java
abstract class Shape {
    public abstract double getPerimeter();
    public abstract double getArea();
}
interface Resizable { void resize(int percent); }
class Circle extends Shape {
    protected double radius;
    public Circle(double radius) {
        if (!Double.isFinite(radius) || radius <= 0)
            throw new IllegalArgumentException("radius > 0");
        this.radius = radius;
    }
    public double getPerimeter() { return 2*Math.PI*radius; }
    public double getArea() { return Math.PI*radius*radius; }
}
class ResizableCircle extends Circle implements Resizable {
    public ResizableCircle(double r) { super(r); }
    public void resize(int percent) {
        if (percent <= 0)
            throw new IllegalArgumentException("percent > 0");
        radius *= percent / 100.0;
        System.out.printf("P=%.2f; S=%.2f%n",
                          getPerimeter(), getArea());
    }
}
```

Với bán kính ban đầu 2, resize(150) làm bán kính thành 3; chu vi = 6π ≈ 18.85, diện tích = 9π ≈ 28.27. Nếu tiếp tục resize(50), bán kính hiện tại 3 giảm còn 1.5, không quay về 1 vì thao tác áp dụng lên kích thước hiện tại.

Đối với Square: lưu side, getPerimeter = 4 × side, getArea = side × side; resize nhân side với percent / 100.0. Với side = 4 và 50%, cạnh mới 2, chu vi 8, diện tích 4. Khi chiều dài nhân k, chu vi nhân k còn diện tích nhân k^2.

Quy tắc percent > 0 là giả định kiểm tra thêm của ví dụ. Đề nêu 100 giữ nguyên, 50 thu nhỏ, 150 phóng lớn. Việc in kết quả trong resize ở đây bám yêu cầu đề; thiết kế ứng dụng lớn có thể tách tính toán và hiển thị.

10.3. Sạp chợ: đa hình ở công thức tính tiền

Nguồn: S9, trang 5-6; bảng thuế đã được đọc từ hình trong PDF.

Dữ liệu chung: số thứ tự, diện tích, doanh thu. Tiền thuê = 40.000.000 × diện tích mỗi năm. Sạp thực phẩm chịu thuế 5% và thêm phí đông lạnh; quần áo 10%; trang sức có các mức 20% và 30% theo phần doanh thu.

Thiết kế: abstract Sap giữ dữ liệu chung và phương thức tienThue(); thueDoanhThu() là abstract; tienDichVu() mặc định 0; tongTien() cộng ba thành phần. SapThucPham ghi đè thuế và dịch vụ; SapQuanAo ghi đè thuế; SapTrangSuc ghi đè thuế theo ngưỡng.

JAVA

```java
// Trong lớp Sap, giả sử doanhThu và dienTich đã hợp lệ.
public double tienThue() { return 40_000_000.0 * dienTich; }
public abstract double thueDoanhThu();
public double tienDichVu() { return 0; }
public double tongTien() {
    return tienThue() + thueDoanhThu() + tienDichVu();
}
```

Điểm cần nêu giả định: thuế trang sức

Đề ghi “Phần doanh thu < 100.000.000 đồng: 20%” và “Phần doanh thu >= 100.000.000 đồng: 30%”. Ví dụ này diễn giải theo phần doanh thu (lũy tiến): 100 triệu đầu tính 20%, phần vượt 100 triệu tính 30%. Nếu lớp quy ước áp một thuế suất lên toàn bộ doanh thu theo ngưỡng, công thức sẽ khác; cần thống nhất cách hiểu khi làm bài nộp.

JAVA

```java
return Math.min(doanhThu, 100_000_000.0) * 0.20
    + Math.max(0, doanhThu - 100_000_000.0) * 0.30;
```

Sạp minh họa	Tiền thuê	Thuế	Dịch vụ	Tổng
Thực phẩm: 2 m², thu 200 triệu	80 triệu	10 triệu	3 triệu	93 triệu
Quần áo: 1 m², thu 200 triệu	40 triệu	20 triệu	0	60 triệu
Trang sức: 1 m², thu 150 triệu	40 triệu	35 triệu	0	75 triệu

Thuế trang sức = 100 × 20% + 50 × 30% = 35 triệu. Tổng ba sạp = 228 triệu. Nếu dùng cách áp 30% toàn bộ 150 triệu thì riêng thuế là 45 triệu, cho thấy vì sao phải ghi rõ giả định.

Cho các đối tượng vào Sap[] và cộng s.tongTien(); lời gọi thueDoanhThu trong hàm chung vẫn chọn công thức của lớp con. Diện tích/doanh thu/phí phải không âm; ví dụ dùng double, không dùng int cho tổng tiền lớn dễ tràn.

10.4. Ca sĩ: tính lương và tìm giá trị lớn nhất

Nguồn: S9, trang 7.

CaSi giữ họ tên, số năm và số đĩa. Có thể cho CaSi là lớp abstract với tinhLuong(), sau đó hai lớp CaSiChuaNoiTieng và CaSiNoiTieng hiện thực hai công thức. CaSiNoiTieng thêm soGameshow.

```java
// Công thức của ca sĩ chưa nổi tiếng:
return 3_000_000L + 500_000L * soNam + 1_000L * soDia;
// Công thức của ca sĩ nổi tiếng:
return 5_000_000L + 500_000L * soNam
    + 1_200L * soDia + 500_000L * soGameshow;
```

Đặt hai return trong hai phương thức khác nhau, mỗi phương thức trả long. Hậu tố L bảo đảm phép nhân/cộng dùng long. Các số đếm phải không âm.

Ví dụ A chưa nổi tiếng: 2 năm, 1000 đĩa → 3.000.000 + 1.000.000 + 1.000.000 = 5.000.000 đồng. B nổi tiếng: 2 năm, 1000 đĩa, 3 gameshow → 5.000.000 + 1.000.000 + 1.200.000 + 1.500.000 = 8.700.000 đồng.

Thuật toán tìm ca sĩ lương cao nhất

```java
static CaSi caoNhat(CaSi[] ds) {
    if (ds == null || ds.length == 0) return null;
    CaSi best = ds[0];
    for (int i = 1; i < ds.length; i++) {
        if (ds[i].tinhLuong() > best.tinhLuong()) {
            best = ds[i];
        }
    }
    return best;
}
```

Giả định mọi phần tử mảng đều khác null. Ban đầu chọn người đầu làm best. Đọc từng người còn lại, nếu lương lớn hơn thì cập nhật. Với A và B, best ban đầu là A có 5 triệu; gặp B có 8,7 triệu thì đổi sang B.

Nếu có nhiều người cùng lương cao nhất, dấu > giữ người xuất hiện đầu tiên. Điều này đáp ứng đề chỉ cần trả một người. Nếu dùng >= sẽ giữ người cuối cùng trong nhóm bằng nhau, cũng phù hợp yêu cầu nhưng nên nhất quán.

Danh sách rỗng trả null; bên gọi phải kiểm tra trước khi in tên. Đừng khởi tạo “người cao nhất” bằng một đối tượng giả rồi vô tình trả đối tượng không có trong danh sách. Thuật toán duyệt một lần: O(n) thời gian, O(1) bộ nhớ phụ nếu tính lương là O(1).

11 / Những điểm cần hiệu chỉnh khi học slide

11.1. Cú pháp và khái niệm dễ bị chép sai

Bảng dưới giúp dùng tài liệu gốc mà không học theo các lỗi đánh máy hoặc các phát biểu quá rộng.

Nguồn	Cần đọc lại chính xác
S1 tr.3	byte là -128..127; int là $-2^{31}..2^{31}-1$; long là $-2^{63}..2^{63}-1$.
S1 tr.3	Không kết luận boolean có kích thước bộ nhớ đúng 1 bit.
S1 tr.6	/ là chia nguyên chỉ khi toán hạng phù hợp kiểu nguyên; có số thực thì chia số thực.
S1 tr.8	int không gọi i.toString(); dùng String.valueOf(i) hoặc Integer.toString(i).
S1 tr.10	printf dùng %d cho số nguyên, không dùng %i.
S2 tr.19	Viết String với S hoa, không phải string.
S3 tr.5-7	Lập trình thủ tục vẫn tái sử dụng mã qua hàm; không phải mọi thay đổi đều phải viết lại toàn bộ.
S3 tr.6	Hàm static không truy cập trực tiếp trường instance khi không có đối tượng; ví dụ cần điều chỉnh.
S4 tr.14	float bi = 3.14 cần hậu tố f, hoặc đổi biến sang double.
S4 tr.20	this.test chỉ trường instance; test không có this ưu tiên biến local đang che tên.
S4 tr.24	Java truyền giá trị, kể cả giá trị tham chiếu; không truyền biến tham chiếu của bên gọi.
S4 tr.28	Constructor không bắt buộc public; constructor sao chép do lập trình viên tự định nghĩa.
S4 tr.34	static có thể truy cập instance qua một đối tượng cụ thể, chỉ không có this ngầm định.
S5 tr.15	Ép kiểu đổi góc nhìn tham chiếu, không biến đối tượng cha bất kỳ thành đối tượng con.
S5 tr.17	has-a chưa đủ khẳng định composition; cần xét sở hữu và vòng đời.
S6 tr.19	Lớp abstract có thể không có phương thức abstract.
S7 tr.3	java.lang.Object không có nhap(); mẫu Object[] là minh họa, cần thay bằng lớp có phương thức phù hợp.

Các dấu ngoặc kép kiểu trình bày “...” trong slide phải đổi thành "..." khi gõ mã. Java phân biệt this/This, new/New và void/Void. Giữ đúng chữ thường của từ khóa.

Các ví dụ trong bản này được diễn giải và rà soát về logic; không coi mọi khối mã rời là một file Java có thể chạy độc lập. Khi ghép, phải đặt đúng lớp, phương thức, import và tránh trùng tên ví dụ.

11.2. Tự kiểm tra và cách ôn theo bài tập

Mười câu hỏi ngắn

1. int a = 5, b = 2; giá trị của (double)(a / b) và (double)a / b có giống nhau không?

2. Vì sao new SinhVien[10] chưa tạo 10 sinh viên?

3. Nếu b = a với a là một đối tượng, có mấy đối tượng được tạo thêm?

4. this.ten = ten phân biệt hai ten như thế nào?

5. Khi truyền đối tượng vào hàm, gán lại tham số có đổi biến bên gọi không?

6. Nguoi n = new SinhVien(...); gọi n.xuat() chạy bản nào nếu con ghi đè?

7. Chỉ đổi kiểu trả về có tạo overload hợp lệ không?

8. Lớp abstract có constructor được không?

9. Hình thoi composition đặt phía toàn thể hay bộ phận?

10. Nếu xóa giữa mảng mà không dịch các phần tử phía sau thì có đúng không?

Đáp án có giải thích

1. Không: lần lượt 2.0 và 2.5. Cách đầu chia nguyên xong mới ép kiểu; cách sau ép trước khi chia.

2. Nó tạo 10 ô tham chiếu null. Cần gán từng ô bằng new SinhVien(...).

3. Không có đối tượng mới; chỉ thêm một biến cùng tham chiếu đối tượng cũ.

4. this.ten là trường của đối tượng hiện tại, ten bên phải là tham số đang che tên.

5. Không. Tham số là bản sao của giá trị tham chiếu; sửa đối tượng chung thì có thể thấy từ bên gọi, gán lại tham số thì không.

6. Bản SinhVien.xuat(), vì đối tượng thật là SinhVien và đây là phương thức instance được ghi đè.

7. Không. Cần khác danh sách kiểu tham số/số tham số/thứ tự kiểu.

8. Có. Constructor chạy trong quá trình tạo đối tượng lớp con; không được new trực tiếp lớp abstract.

9. Phía toàn thể/chủ sở hữu. Tam giác rỗng của kế thừa lại chỉ về lớp cha.

10. Không theo cách quản lý mảng liên tục đang dùng: cần lấp ô bị xóa và cập nhật kích thước; chỉ thu ngắn sẽ làm mất phần tử cuối thay vì phần tử mục tiêu.

Một vòng học thực hành

Lần 1: tự viết chương trình nhập hai số và tính toán, thêm trường hợp chia cho 0. Lần 2: mảng số với tìm, thêm, xóa. Lần 3: Diem và HinhChuNhat có constructor, getter/setter. Lần 4: Nguoi/SinhVien có kế thừa. Lần 5: mảng HinhHoc gọi đa hình. Lần 6: ghép danh sách sinh viên có đủ CRUD, rồi tự vẽ UML khớp với mã.

Mỗi lần chỉ nên thay một yếu tố và dự đoán kết quả trước khi chạy. Nếu chưa giải thích được vì sao kết quả đúng, quay lại theo dõi giá trị từng biến hoặc đối tượng mà từng tham chiếu đang trỏ tới.